

CSE563 — Assignment 6

Unit VI: Algorithmic and Optimization Approaches

Maps to CO6 (discuss different optimization approaches to solve mathematical problems). **Marks: 50.** Attempt all questions. Show DP tables in full and state the traceback.

Part A — Short Answer and Definitions (10 marks, 2 each)

Q1. [2 marks] State the two properties a problem must have for a greedy algorithm to be correct. Give one problem where greedy works and one where it provably fails.

Solution. (i) **Greedy-choice property:** a locally optimal choice is contained in some globally optimal solution (proved by an exchange argument). (ii) **Optimal substructure:** an optimal solution to the whole contains optimal solutions to the subproblems.

Works: fractional knapsack — taking the item with the highest value/weight ratio is always safe, because any leftover capacity can be filled with a fraction. **Fails:** 0/1 knapsack — with capacity 7 and items (3, 4), (4, 5), (5, 7), greedy by ratio takes (5, 7) first (ratio 1.4) and can then add nothing, giving value 7, whereas (3, 4) + (4, 5) fits exactly and gives 9.

Q2. [2 marks] Explain the difference between dynamic programming and divide-and-conquer. Which property decides between them?

Solution. Both rely on **optimal substructure** and both break a problem into subproblems. The deciding property is whether the subproblems **overlap**.

Divide and conquer: subproblems are *disjoint*, each solved once (merge sort's two halves never share an element). Recursion is efficient. **Dynamic programming:** subproblems *recur*, so naive recursion re-solves them exponentially often. DP stores each subproblem's answer once (memoisation or tabulation) and reuses it, collapsing exponential work to polynomial — e.g. Fibonacci from $\Theta(\varphi^n)$ to $\Theta(n)$.

Q3. [2 marks] Define backtracking and branch and bound, and state the difference between them.

Solution. **Backtracking** is a depth-first search over partial solutions: extend a partial solution by one decision, and as soon as a constraint is violated, abandon (*prune*) that branch and undo the last decision. It is used for *feasibility* / constraint-satisfaction problems (*N*-Queens, Sudoku, graph colouring).

Branch and bound is the *optimisation* variant: in addition to pruning infeasible branches, it computes a **bound** on the best value obtainable in a branch, and prunes any branch whose bound cannot beat the best complete solution found so far (the *incumbent*).

Difference: backtracking prunes on *infeasibility*; branch and bound also prunes on *sub-optimality*.

Q4. [2 marks] Name the three genetic operators and state the role of each in balancing exploration and exploitation.

Solution.

- **Selection** — fitter chromosomes get more copies in the mating pool (roulette wheel, tournament, rank). This is **exploitation**: it concentrates the search around what already

works.

- **Crossover** — children combine genetic material from two parents. This is *recombinative exploration*: it explores new points, but only within the material already present in the population.
- **Mutation** — random small changes to individual genes. This is pure **exploration**: it reintroduces lost alleles and is the only operator that can generate genuinely new material, preventing premature convergence.

Too much selection pressure with too little mutation causes the population to converge prematurely on a local optimum.

Q5. [2 marks] State the No Free Lunch theorem and explain its practical implication for choosing an optimisation algorithm.

Solution. No Free Lunch (Wolpert & Macready, 1997): averaged over *all* possible objective functions, every optimisation algorithm has identical expected performance. No algorithm is universally superior.

Practical implication: algorithm choice must be justified by the *structure of the problem at hand*, not by the general reputation of the method. If the problem has exploitable structure (convexity, a matroid, optimal substructure), use the exact algorithm that exploits it — never a genetic algorithm to sort a list or find a shortest path. Metaheuristics earn their place only when the problem is NP-hard, large, and lacks such structure; and even then, different metaheuristics suit different landscapes.

Part B — Exact Algorithm Design (26 marks)

Q6. [6 marks] Nine activities with (start, finish) times: (1, 4), (3, 5), (0, 6), (5, 7), (3, 8), (5, 9), (6, 10), (8, 11), (12, 16). (a) State the greedy rule and justify it in one sentence. (b) Apply it, showing the selected set. (c) Show that the rule “pick the activity with the shortest duration” can be sub-optimal by constructing a small counterexample.

Solution. (a) Rule: sort by **finish time** and repeatedly select the compatible activity that finishes earliest. Justification: finishing earliest leaves the maximum remaining time for all subsequent activities, so it is never worse than any other first choice (exchange argument: swapping the earliest-finishing activity into any optimal solution keeps it feasible and the same size).

(b) Sorted by finish: (1, 4), (3, 5), (0, 6), (5, 7), (3, 8), (5, 9), (6, 10), (8, 11), (12, 16).

Activity	Start $\geq$ last finish?	Decision
(1, 4)	—	select , last finish = 4
(3, 5)	$3 < 4$	reject
(0, 6)	$0 < 4$	reject
(5, 7)	$5 \geq 4$	select , last finish = 7
(3, 8)	$3 < 7$	reject
(5, 9)	$5 < 7$	reject
(6, 10)	$6 < 7$	reject
(8, 11)	$8 \geq 7$	select , last finish = 11
(12, 16)	$12 \geq 11$	select

Selected: (1, 4), (5, 7), (8, 11), (12, 16) — **four activities**.

(c) Shortest-duration counterexample. Take three activities: (0, 5), (4, 6), (5, 10). The

shortest is (4, 6) (duration 2); selecting it blocks both others (it overlaps (0, 5) and (5, 10) — the latter only if intervals are closed; using (4, 6) and (5, 10) overlap on [5, 6]), giving only **1** activity. The optimal solution is (0, 5) and (5, 10) — **2** activities. So shortest-duration-first is not optimal, while earliest-finish-first correctly picks (0, 5) then (5, 10).

Q7. [7 marks] Solve the 0/1 knapsack instance with capacity $W = 8$ and items $I_1 = (w=2, v=3)$, $I_2 = (3, 4)$, $I_3 = (4, 5)$, $I_4 = (5, 8)$. (a) Build the full DP table $\text{dp}[i][c]$ for $i = 0..4$, $c = 0..8$. (b) State the optimal value. (c) Trace back to identify which items are chosen and verify the weight fits. (d) State the time complexity and explain why it is called *pseudo-polynomial*.

Solution. (a) Recurrence: $\text{dp}[i][c] = \max(\text{dp}[i-1][c], \text{dp}[i-1][c - w_i] + v_i)$ when $w_i \leq c$, else $\text{dp}[i-1][c]$.

$i \backslash c$	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1 (2,3)	0	0	3	3	3	3	3	3	3
2 (3,4)	0	0	3	4	4	7	7	7	7
3 (4,5)	0	0	3	4	5	7	8	9	9
4 (5,8)	0	0	3	4	5	8	8	11	12

Sample cells. $\text{dp}[2][5] = \max(\text{dp}[1][5], \text{dp}[1][2] + 4) = \max(3, 3 + 4) = 7$. $\text{dp}[3][8] = \max(\text{dp}[2][8], \text{dp}[2][4] + 5) = \max(7, 4 + 5) = 9$. $\text{dp}[4][8] = \max(\text{dp}[3][8], \text{dp}[3][3] + 8) = \max(9, 4 + 8) = 12$.

(b) **Optimal value** = $\text{dp}[4][8] = \mathbf{12}$.

(c) **Traceback.** $\text{dp}[4][8] = 12 \neq \text{dp}[3][8] = 9 \Rightarrow$ **item 4 taken**; capacity $8 - 5 = 3$. $\text{dp}[3][3] = 4 = \text{dp}[2][3] \Rightarrow$ item 3 not taken. $\text{dp}[2][3] = 4 \neq \text{dp}[1][3] = 3 \Rightarrow$ **item 2 taken**; capacity $3 - 3 = 0$. $\text{dp}[1][0] = 0 \Rightarrow$ item 1 not taken.

Chosen: I_2 and I_4 . Weight = $3 + 5 = 8 \leq 8 \checkmark$; value = $4 + 8 = 12 \checkmark$. (Greedy by ratio would rank I_1 at 1.5, I_4 at 1.6, I_2 at 1.33, I_3 at 1.25; taking I_4 then I_1 gives weight 7, value 11 — worse than 12.)

(d) **Time complexity** $\Theta(nW)$ — one constant-time cell per (item, capacity) pair. It is **pseudo-polynomial** because W is a *numeric value*, not the input size. Writing W in binary takes only $\log_2 W$ bits, so $\Theta(nW) = \Theta(n2^{\log_2 W})$ is exponential in the length of the encoded input. That is why 0/1 knapsack remains NP-complete despite this table algorithm.

Q8. [6 marks] For $X = \text{AGGTAB}$ and $Y = \text{GXTXAYB}$: (a) Build the LCS dynamic-programming table in full. (b) State the LCS length. (c) Trace back to recover the actual subsequence. (d) State the time and space complexity, and name one real application.

Solution. (a) $L[i][j] = L[i-1][j-1] + 1$ if $X_i = Y_j$, else $\max(L[i-1][j], L[i][j-1])$.

	$\emptyset$	G	X	T	X	A	Y	B
$\emptyset$	0	0	0	0	0	0	0	0
A	0	0	0	0	0	1	1	1
G	0	1	1	1	1	1	1	1
G	0	1	1	1	1	1	1	1
T	0	1	1	2	2	2	2	2
A	0	1	1	2	2	3	3	3
B	0	1	1	2	2	3	3	4

(b) **LCS length** = 4.

(c) **Traceback** from $L[6][7]$ (row = $X_6 = \text{B}$, column = $Y_7 = \text{B}$):

- $\text{B} = \text{B} \Rightarrow$ take **B**, move diagonally to $L[5][6]$.

- $X_5 = A, Y_6 = Y$: not equal. $L[4][6] = 2, L[5][5] = 3$; move *left* to $L[5][5]$.
- $X_5 = A, Y_5 = A \Rightarrow$ take **A**, move diagonally to $L[4][4]$.
- $X_4 = T, Y_4 = X$: not equal. $L[3][4] = 1, L[4][3] = 2$; move *left* to $L[4][3]$.
- $X_4 = T, Y_3 = T \Rightarrow$ take **T**, move diagonally to $L[3][2]$.
- $X_3 = G, Y_2 = X$: not equal. $L[2][2] = 1, L[3][1] = 1$; move *up* to $L[2][2]$.
- $X_2 = G, Y_2 = X$: not equal; move to $L[2][1]$.
- $X_2 = G, Y_1 = G \Rightarrow$ take **G**, move to $L[1][0]$ — boundary, stop.

Collected in reverse order: B, A, T, G $\Rightarrow$ **LCS = GTAB**. Check: GTAB appears in AGGTAB and in GXTXAYB. ✓

(d) Time $\Theta(mn)$, space $\Theta(mn)$ (reducible to $\Theta(\min(m, n))$ if only the length is needed, since each row depends on the previous row alone — but the traceback then requires Hirschberg's algorithm). **Applications:** the **diff** utility and version-control merge (git), DNA/protein sequence alignment in bioinformatics, and plagiarism detection.

Q9. [7 marks] (a) Explain the backtracking approach to the N -Queens problem and state the pruning conditions used. (b) Trace the search for $N = 4$ from an empty board, listing each partial placement and whether it is extended or pruned. (c) Give both solutions for $N = 4$. (d) Compare the number of nodes explored with brute force over all 4^4 placements.

Solution. (a) Place one queen per row, proceeding row by row. Represent a partial solution as a list of column indices $[c_0, c_1, \dots]$. Before placing a queen at column c in row r , check it against every queen already placed at (r', c') :

- **Same column:** $c = c' \Rightarrow$ reject.
- **Same diagonal:** $|r - r'| = |c - c'| \Rightarrow$ reject.

(No row check is needed — the representation places exactly one queen per row by construction.) If no column in a row is valid, backtrack: undo the previous row's placement and try its next column.

(b) Trace for $N = 4$ (columns numbered 0–3):

Partial	Outcome
[0]	row 1: col 0 same column $\times$; col 1 diagonal $\times$; col 2 valid
[0, 2]	row 2: col 0 diagonal from (1, 2)? $ 2 - 1 = 1 = 0 - 2 $? no, $= 2$ — but col 0 shares a column with (0, 0) $\times$; col 1 diagonal from (1, 2) $\times$; col 2 same column $\times$; col 3 diagonal from (1, 2) $\times \Rightarrow$ prune
[0, 3]	row 1 retried with col 3: valid. row 2: col 1 valid
[0, 3, 1]	row 3: col 0 $\times$ (column 0 taken); col 1 $\times$; col 2 $\times$ (diagonal from (2, 1)); col 3 $\times \Rightarrow$ prune ; row 2 has no further columns $\Rightarrow$ backtrack; row 1 exhausted $\Rightarrow$ backtrack to row 0
[1]	row 1: col 0 diag $\times$; col 1 col $\times$; col 2 diag $\times$; col 3 valid
[1, 3]	row 2: col 0 valid
[1, 3, 0]	row 3: col 2 valid
[1, 3, 0, 2]	Solution 1
[2]	by left–right symmetry, yields Solution 2

(c) The two solutions for $N = 4$:

[1, 3, 0, 2]	[2, 0, 3, 1]

The two are mirror images of each other.

(d) Brute force over all column assignments examines $4^4 = \mathbf{256}$ boards (or $4! = 24$ if one queen

per column is assumed). The backtracking search above examined roughly **a dozen** partial boards, because pruning eliminates an entire subtree the moment a conflict appears. The saving grows dramatically with N : for $N = 8$ brute force is $8^8 \approx 1.7 \times 10^7$ while backtracking explores on the order of 2×10^3 nodes to find all 92 solutions.

Part C — Metaheuristics (14 marks)

Q10. [8 marks] A genetic algorithm maximises $f(x) = x^2$ for $x \in [0, 31]$ using 5-bit binary encoding. The initial population is $P_1 = 01100$, $P_2 = 11001$, $P_3 = 00101$, $P_4 = 10010$. (a) Decode each chromosome and compute its fitness. (b) Compute the selection probability and expected number of copies under roulette-wheel selection. (c) Perform single-point crossover between (P_2, P_4) at position 2 and between (P_2, P_1) at position 3, and decode the children. (d) Apply a mutation flipping bit 1 (the most significant bit) of the first child, and give its new fitness. (e) Comment on whether the generation improved the population.

Solution. (a) **Decode and evaluate.**

Chromosome	Binary	x	$f(x) = x^2$
P_1	01100	12	144
P_2	11001	25	625
P_3	00101	5	25
P_4	10010	18	324
$\sum f =$			1118

Average fitness = $1118/4 = 279.5$; best = 625.

(b) **Roulette-wheel selection.** $p_i = f_i / \sum f$; expected copies = $p_i N$ with $N = 4$.

Chromosome	p_i	Expected copies
P_1	$144/1118 = 0.129$	0.52
P_2	$625/1118 = 0.559$	2.24
P_3	$25/1118 = 0.022$	0.09
P_4	$324/1118 = 0.290$	1.16

P_2 is expected to contribute about two copies, P_4 one, and P_3 (fitness 25) is very likely to be eliminated. Mating pool $\approx \{P_2, P_2, P_4, P_1\}$.

(c) **Crossover.** Pair (P_2, P_4) , single point after bit 2: 11|001 and 10|010

$$\rightarrow C_1 = 11010 = \mathbf{26} \ (f = 676), \quad C_2 = 10001 = \mathbf{17} \ (f = 289).$$

Pair (P_2, P_1) , single point after bit 3: 110|01 and 011|00

$$\rightarrow C_3 = 11000 = \mathbf{24} \ (f = 576), \quad C_4 = 01101 = \mathbf{13} \ (f = 169).$$

(d) **Mutation on C_1 .** Flip the most significant bit of 11010: $\rightarrow 01010 = 10$, giving $f = \mathbf{100}$. This mutation is *harmful* — it destroyed the best individual, dropping fitness from 676 to 100. That is why mutation rates are kept low ($p_m \approx 0.001$ – 0.05) and why **elitism** (copying the best individual through unchanged) is normally used.

(e) **Did the generation improve?** Without the mutation, the new population $\{26, 17, 24, 13\}$ has fitnesses $\{676, 289, 576, 169\}$: sum 1710, average 427.5 (up from 279.5), best 676 (up from 625). **Yes — both the average and the best improved.** With the harmful mutation applied, the population is $\{10, 17, 24, 13\}$ with sum 1134, average 283.5, best 576 — barely better than the start and the best individual is *worse*. The comparison demonstrates the value of elitism.

Q11. [6 marks] (a) Write the simulated-annealing acceptance rule and compute the acceptance probability of a move with $\Delta = +5$ at $T = 10$, $T = 2$ and $T = 0.5$ (use $e^{-0.5} = 0.6065$, $e^{-2.5} = 0.0821$, $e^{-10} = 4.54 \times 10^{-5}$). (b) Explain, using these numbers, how the algorithm shifts from exploration to exploitation. (c) Write the PSO velocity update and perform one update for a particle with $x = 3$, $v = -1$, personal best $p = 5$, global best $g = 8$, $w = 0.7$, $c_1 = c_2 = 2$, $r_1 = 0.4$, $r_2 = 0.6$.

Solution. (a) **Acceptance rule.** For a candidate move changing the cost by $\Delta = C(s') - C(s)$ (minimisation):

$$P(\text{accept}) = \begin{cases} 1 & \Delta \leq 0 \quad (\text{downhill} \text{ — always accept}) \\ e^{-\Delta/T} & \Delta > 0 \quad (\text{uphill} \text{ — accept with this probability}) \end{cases}$$

For $\Delta = +5$:

T	Δ/T	$P = e^{-\Delta/T}$
10	0.5	0.6065
2	2.5	0.0821
0.5	10	4.54×10^{-5}

(b) At $T = 10$ a move that worsens the cost by 5 is still accepted about 61% of the time: the search wanders almost freely and can climb out of any local minimum — this is **exploration**. At $T = 2$ the same move is accepted only 8% of the time, so the search mostly descends but retains a small escape probability. At $T = 0.5$ acceptance is effectively zero (0.0045%): the algorithm has become pure hill-climbing, refining the basin it currently occupies — **exploitation**. The cooling schedule $T \leftarrow \alpha T$ (with $\alpha \approx 0.9\text{--}0.99$) therefore performs a continuous, automatic handover from exploration to exploitation.

(c) **PSO update.**

$$v \leftarrow wv + c_1 r_1 (p - x) + c_2 r_2 (g - x), \quad x \leftarrow x + v.$$

Substituting:

$$\begin{aligned} v &= 0.7(-1) + 2(0.4)(5 - 3) + 2(0.6)(8 - 3) \\ &= -0.7 + 0.8(2) + 1.2(5) \\ &= -0.7 + 1.6 + 6.0 = \mathbf{6.9} \\ x &= 3 + 6.9 = \mathbf{9.9} \end{aligned}$$

The particle was moving away ($v = -1$) but the cognitive term (+1.6, pulling toward its own best at 5) and especially the social term (+6.0, pulling toward the swarm best at 8) reverse it. Note the particle overshoots past $g = 8$ to 9.9 — this overshoot is exactly what lets PSO explore beyond the current best rather than merely converging onto it.